

Yinsh Game

DONE BY:

AMR M. ELKHODARI

MOHAMED I. SOLTAN

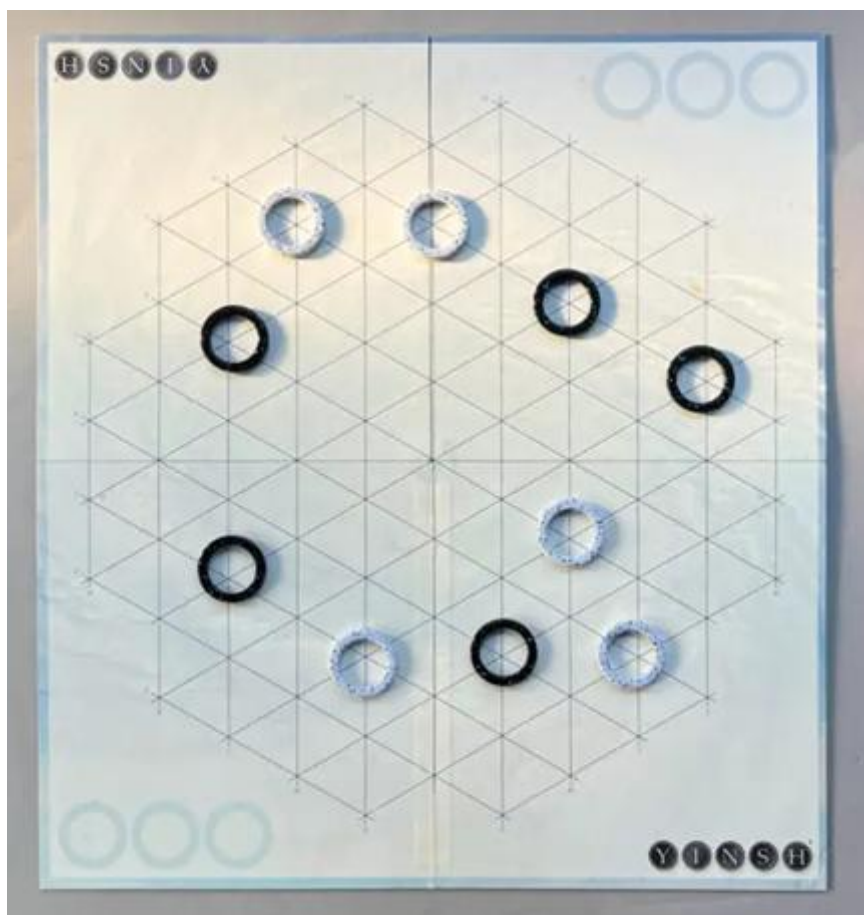
FARES K. IBRAHIM

SUPERVISED BY:

ENG/MAI MOHAMED

Project Idea

It is a game played on a hexagonal board. Players place rings and then drop markers inside them. When a marker is dropped, the ring moves in a straight line, flipping the colors of all markers it jumps over. If you get five markers of your color in a row, you remove them *and* one of your rings. The first player removes three of their rings' wins.



PEAS Description

- **P - Performance Measure:** What defines "success" for the agent?
 - *Primary:* Winning the game (removing 3 of its own rings before the opponent does).
 - *Secondary:* Maximizing the number of its own markers on the board (controlling the economy) and minimizing the opponent's markers.
 - *Algorithmic:* Executing moves within a reasonable time limit (efficiency in searching the state-space).
- **E - Environment:** The world the agent operates in.
 - The hexagonal YINSH board.
 - The playing pieces: 5 rings per player, and a shared pool of black and white markers.
 - The opponent.
- **A - Actuators:** How the agent acts upon the environment.
 - In a software project, these are the functions that execute moves on the digital board.
 - *Actions include:* Placing a ring (Phase 1), dropping a marker inside a ring, moving a ring along an axis, flipping jumped markers, and removing a row of 5 markers along with a ring.
- **S - Sensors:** How the agent receives information from the environment.
 - The input of the current board state.
 - *Sensory data includes:* The exact coordinate arrays of all rings and markers, the current score (rings removed by each player), and receiving the opponent's move.

ODESDA Description

- ❖ **O - Observable (Fully Observable):** The agent always has access to the complete state of the board. There is no hidden information or hidden cards.
- ❖ **D - Deterministic (Strategic):** If the agent makes a specific move, the resulting board state is 100% guaranteed. There are no dice rolls or random chances.
- ❖ **E - Episodic (Sequential):** It is highly **Sequential**. The agent's current move heavily impacts all future moves. You cannot evaluate a single turn in isolation (an episode); you must look ahead down the game tree.
- ❖ **S - Static (Static):** The environment does not change while the agent is "thinking." The board waits for the agent to calculate its algorithm.
- ❖ **D - Discrete (Discrete):** There are a finite number of hexes on the board, a finite number of pieces, and a distinct, finite set of legal moves at any given time. Time flows in discrete turns.
- ❖ **A - Agents (Multi-agent / Two-player competitive):** The agent is not alone. It operates in a zero-sum, strictly competitive environment against an opponent who is actively trying to minimize the agent's performance measure.

Agent Type

Simple-Reflex Agent? No. *Why not:* A simple-reflex agent only acts on the *current* percept using pre-programmed condition-action rules. It has no memory and does not project into the future. YINSH is far too complex for simple if/else logic.

Model-Based Reflex Agent? No. *Why not:* A model-based agent's main strength is maintaining an internal state to track things it cannot currently see (handling *partial observability*). Since YINSH is fully observable, this architecture adds no value. It still lacks the ability to look ahead and simulate future outcomes.

Goal-Based Agent? No. *Why not:* A goal-based agent can search into the future to find a sequence of actions that leads to a goal. However, it treats all paths to the goal equally. In YINSH, the state-space is too massive to search all the way to the winning state on every turn. The agent needs to know not just *if* a state is a goal, but *how good* a non-goal state is compared to another.

Utility-Based Agent? Yes. This is the correct choice. *Why:* Because the agent usually cannot search the game tree all the way to the end (due to time and memory constraints), it must stop searching at a certain depth and evaluate the board. A utility-based agent uses a **heuristic evaluation function** (a utility function) to map a state to a real number indicating how "happy" the agent is with that board state (e.g., scoring highly for controlling the center or having multiple 3-in-a-rows). It then uses algorithms to maximize its own utility while assuming the opponent will minimize it.